

④ 字符串处理指令

字符串操作指令实质是对一片连续存储单元进行处理,这片存储单元由隐含指针 $DS:SI$ 或 $ES:DI$ 指定.字符串操作指令可对内存单元按字节或字进行处理,并可依操作对象的字节数使变址寄存器 $SI(DI)$ 增减1或2

当 $DF=0$: $SI(DI)$ 自动加1(字节操作)或加2(字操作),字符串操作按"低地址→高地址"正向顺序

当 $DF=1$: $SI(DI)$ 自动减1(字节操作)或减2(字操作),字符串操作按"高地址→低地址"反向顺序

(1) 重复前缀指令 REP

重复其后的字符串操作指令,重复次数由 CX 决定

REP MOVSB $\rightarrow$ 批量传送字符串

REP STOS $\rightarrow$ 批量往内存写同一个数据

REP LODS $\rightarrow$ 批量从内存读数据到寄存器

执行步骤:

判断 $CX \neq 0$

若 $CX=0$, 则结束重复操作,执行下条指令

若 $CX \neq 0$, 则 $CX=CX-1$ (不影响有关标志位)并执行其后字符串操作指令(MOVS/STOS/LODS)

该指令执行完后转回判断 $CX \neq 0$ 处

(2) 字符串传送指令

把以 SI 为指针的源串中的一个字节(字)存储单元中的数据传送到

以 DI 为指针的目的地址,并自动修改指针使之指向下一个字节(字)存储单元

MOVSB 目的操作数,源操作数 (需在操作数中表明数据格式)

MOVSB (字节)

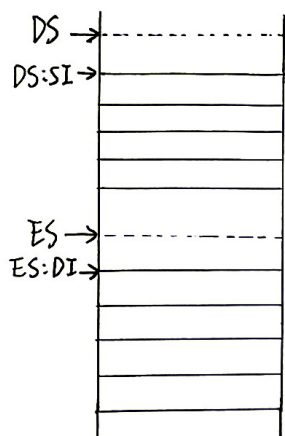
MOVSW (字)

第1步: $(DS:[SI]) \rightarrow ES:[DI]$ 或

第2步: 当 $DF=0$ 时 $(SI)+1/2 \rightarrow (SI)$ $(DI)+1/2 \rightarrow (DI)$

当 $DF=1$ 时 $(SI)-1/2 \rightarrow (SI)$ $(DI)-1/2 \rightarrow (DI)$

REP MOVSB 将数据段中的整串数据传送至附加段中

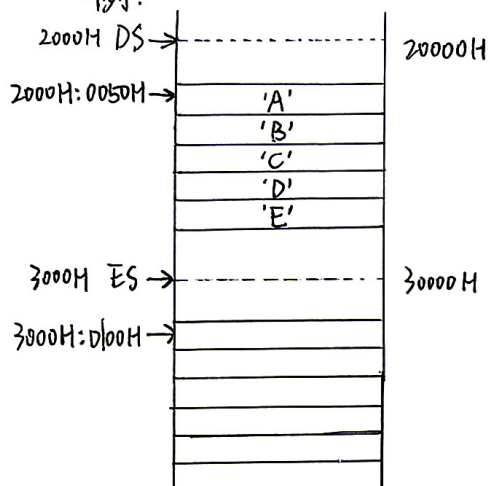


源串(数据段) → 目的串(附加段)

执行 REP MOVSB 前, 应先做好:

{ 源串首地址(末地址) → SI
目的串首地址(末地址) → DI
串长度 → CX
建立方向标志 (CLD使DF=0, STD使DF=1)

例.



MOV SI, 0050H

MOV DI, 0100H

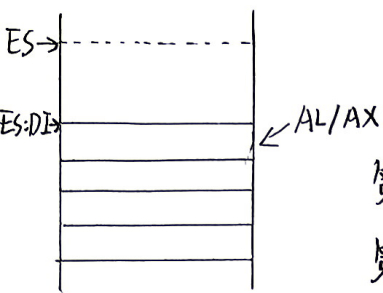
MOV CX, 5

CLD

REP MOVSB

(3) 置字符串数据指令 (把寄存器中数据写入内存)

把 AL 或 AX 中的数据送入 ES:DI 所指的字符串中的字节(字)存储单元中



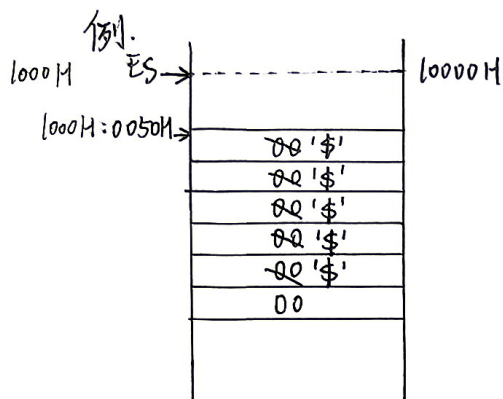
STOS 目的操作数 (需在操作数中表明数据格式)

STOSB ; (字节)

STOSW ; (字)

第1步: 字节操作(AL) → [DI] 字操作(AX) → [DI]

第2步: 修改指针DI使之指向串中的下一个元素(DF=0时,(DI)增; DF=1时,(DI)减)



AL
'\$'

MOV DI, 0050H

MOV AL, '\$'

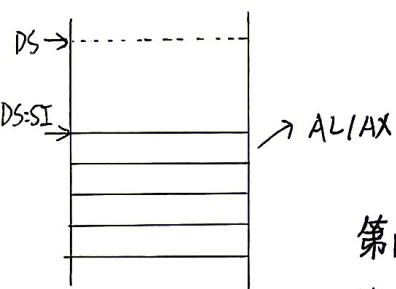
MOV CX, 5

CWD

REP STOSB

(4) 取字符串数据指令 一般不加重复前缀!

把 SI 所指的源串中的一个字节(或字)存储单元中的数据取出来送入 AL(或 AX)中



LODS 源操作数 (需在操作数中表明数据格式)

LODSB ; (字节)

LODSW ; (字)

第1步: 字节操作([SI]) → AL 字操作([SI]) → AX

第2步: 修改指针SI使之指向串中的下一个元素(DF=0时,(SI)增; DF=1时,(SI)减)

(5) 串输入指令 **INS** (从外设读数据写入内存)

把由 **DX** 寄存器指定的 I/O 端口 中的字/字节传送到附加段的 目的串 中, 并根据 **DF** 和数据类型来改变目的变址寄存器的方向内容

INSB ; 字节串输入 $((DX)) \rightarrow ES:[DI]$ $DI \pm 1 \rightarrow DI$

INSW ; 字串输入 $((DX)) \rightarrow ES:[DI]$ $DI \pm 2 \rightarrow DI$

(6) 串输出指令 **OUTS**

把源串中的字/字节传送到由 **DX** 寄存器指定的 I/O 端口 中, 并根据 **DF** 和数据类型来改变源变址寄存器的方向内容

OUTB ; 字节串输出 $DS:[SI] \rightarrow ((DX))$ $SI \pm 1 \rightarrow SI$

OUTW ; 字串输出 $DS:[SI] \rightarrow ((DX))$ $SI \pm 2 \rightarrow SI$

(7) 相等重复前缀指令 REPE / REPZ

REPE / REPZ CMPS / SCAS

执行步骤:

判断 $CX \geq 0$ 或 $ZF = 0$

若条件成立, 则结束重复操作转而执行下条指令

若条件不成立, 则 $CX = CX - 1$ (不影响有关标志位) 并执行其后的字符串操作指令 (CMPS / SCAS), 该指令执行完后再转至“判断”处

(8) 不等重复前缀指令 REPNE / REPNZ

REPNE / REPNZ CMPS / SCAS

执行步骤:

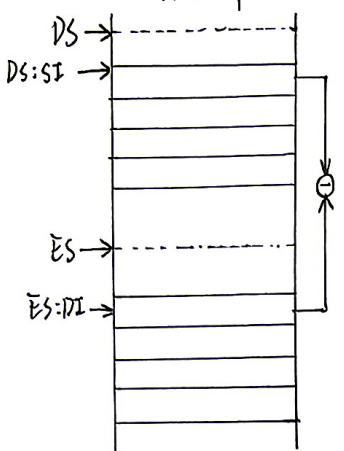
判断 $CX \geq 0$ 或 $ZF = 1$

若条件成立, 则结束重复操作转而执行下条指令

若条件不成立, 则 $CX = CX - 1$ (不影响有关标志位) 并执行其后的字符串操作指令 (CMPS / SCAS), 该指令执行完后再转至“判断”处

(9) 串比较指令 CMPS

把 SI 所指的源串中的一个字节/字存储在单元中的数据与 DI 所指的串中的一个字节/字相减，并依相减结果设置标志，结果不保存



CMPS 源操作数, 目的操作数

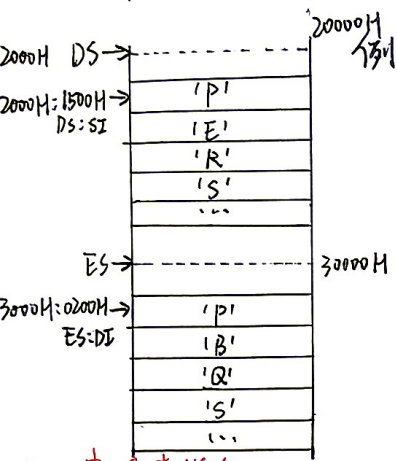
CMPSB ; 字节串比较

CMPSW ; 字串比较

第1步: $[SI] - [DI]$

第2步: 修改串指针使之指向串中下元素

DF=0 时, (SI)、(DI) 增; DF=1 时, (SI)、(DI) 减



例. 比较两个字符串, 找出它们不匹配的位置

MOV SI, 1500H

MOV DI, 0200H

MOV CX, 17

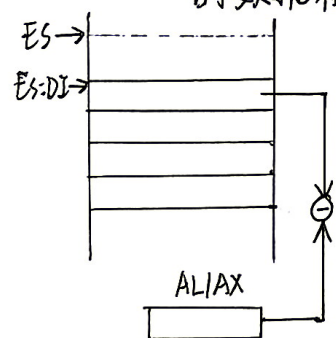
CWD

REPE CMPSB

相等则继续

(10) 串搜索指令 SCAS

把 AL (字节) 或 AX (字) 中的内容与 DI 所指的串中的一个字节/字存储在单元中的数据相减, 依相减结果设置标志位但不保存结果



SCAS 目的操作数

SCASB ; 字节串搜索

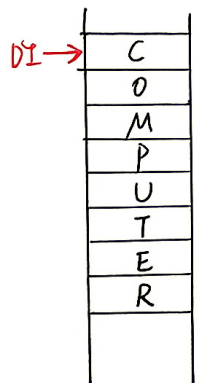
SCASW ; 字串搜索

第1步: $(AL) - [DI]$ 字 $(AX) - [DI]$ 字节

第2步: 修改指针使之指向串中下元素

DF=0 时, (DI) 增; DF=1 时, (DI) 减

例. 从一个字符串中查找一个指定字符



标签, 标记数据段起始偏移地址
MESS DB 'COMPUTER'

LEA DI, MESS ; 把有效地址加载到目标变址寄存器

MOV AL, 'T'

MOV CX, 8

CWD

REPNE SCASB

(CX)=2

不相等则继续

小结

串处理指令 MOVS/MOVSW 、 STOS/STOSW 、 LODS/LODSW 、 CMPS/CMPSW 、 SCAS/SCASW

用于处理连续存储单元中的字节/字操作数

- 与串传送指令 MOVS 、串存入指令 STOS 联用的重复前缀是 REP (取串指令 LODS 一般不加)
与串比较指令 CMPS 、串扫描指令 SCAS 联用的重复前缀是 $\text{REPE}(\text{REPZ})$ 或 $\text{REPNE}(\text{REPNZ})$
- 一般分两步执行, 第1步传送/存取/比较 (处理功能), 第2步修改指针指向下一个要处理的字节/字
- 源串必须在数据段中, 目的串必须在附加段中
允许使用段跨越前缀指定段
- 串处理指令隐含的寻址方式是 SI 、 DI 寄存器的间接寻址方式
- MOVS 、 STOS 、 LODS 指令不影响条件标志位, CMPS 、 SCAS 指令依比较结果设定
- 串处理的方式取决于方向标志 DF

$\text{DF}=0$ 时, 地址指针 SI 、 DI 增量 (+1/+2); $\text{DF}=1$ 时, 地址指针 SI 、 DI 减量 (-1/-2)

可用 CLD 、 STD 指令来设置方向标志位是 0 还是 1.

⑥ 控制转移类指令

无条件转移指令: JMP、子程序的调用和返回指令、中断的调用和返回指令

有条件转移指令

- 基于无符号数的条件转移指令
- 基于有符号数的条件转移指令
- 基于特殊算术标志位的条件转移指令

用于实现分支、循环、过程等程序结构, 通过改变 IP (和 CS) 的值实现程序执行顺序的改变

(1) 无条件转移指令 JMP

段内短转移

```
CODE SEGMENT
:
JMP SHORT NEXT
:
NEXT:
:
CODE ENDS
```

段内近转移

```
CODE SEGMENT
:
JMP NEAR PTR PROGA
:
PROGA:
:
CODE ENDS
```

段间远转移

```
CODE1 SEGMENT
:
JMP FAR PTR NEXT
:
CODE1 ENDS
:
CODE2 SEGMENT
:
NEXT:
:
CODE2 ENDS
```

目标地址可用除立即寻址方式外任何寻址方式表示

相对当前 IP 值的位移量: -128 字节 ~ +127 字节

16 位带符号补码数
-32768 ~ +32767 字节, 可寻址段内任一位置

段长 64KB (65536 字节)

只支持短转移的直接寻址方式

(2) 条件转移指令

若指定的条件 XX 成立, 则程序转到由标号 LABEL 指定的目标地址执行指令;
若指定的条件 XX 不成立, 则程序顺序执行下条指令

所有条件转移指令都是短转移指令, 转移的目标地址必须在当前 IP 地址的 -128 ~ +127 字节范围内

因此, 条件转移指令是 2 字节指令 (1 字节操作码 + 1 字节偏移量)

Jcc 指令不影响标志, 但要利用标志

根据利用的标志位不同, 可分为以下 4 种情况:

- 判断单个标志位状态
- 比较无符号数高低
- 比较有符号数大小
- 测试 CX 值为 0 则转移

16 条 Jcc 指令有 30 个助记符

情况1: 根据单个条件标志的设置情况转移

例. 将AX中存放的无符号数除以2, 若是奇数则加1后除以2

如何判断奇偶?

↓

AX最低位为0就是偶数

AX最低位为1就是奇数

↓

方法1: 用AND将除最低位外的其它位置0

判断这个数是不是0

方法2: 将最低位用移位指令移至进位标志

判断进位标志是不是0

方法3: 将最低位用移位指令移至最高符号位

判断符号标志是不是0

```
TEST AX, 01H
JZ EVEN
ADD AX, 1
EVEN:
SHR AX, 1
```

跳转时 ZF=1

可能造成符号错误
用ROR或SAR

```
MOV BX, AX
SHR BX, 1
JNC EVEN
ADD AX, 1
EVEN:
SHR AX, 1
```

跳转时 CF=0

```
MOV BX, AX
ROR BX, 1
ADD BX, 0
JNS EVEN
ADD AX, 1
EVEN:
SHR AX, 1
```

跳转时 SF=0

ROR不影响SF标志

例. 判断是否为字母Y, 寄存器AL中是字母Y (含大小写), 则令AH=0, 否则令AH=-1

```
CMP AL, 'y'
```

```
JE NEXT
```

```
CMP AL, 'Y'
```

```
JE NEXT
```

```
MOV AH, -1
```

```
JMP DONE
```

```
NEXT:
```

```
MOV AH, 0
```

```
DONE:
```

```
...
```

例. (计数器为0的判断) 复制100个字节的字符串

```
MOV CX, 100
AGAIN:
JCXZ NEXT
MOVSB ; 串字节传送指令
DEC CX
JMP AGAIN
NEXT:
...
```

例. 若 $X > 50$, 转到 TOO-HIGH, 否则 $|X - Y| \rightarrow \text{RESULT}$, 若溢出则转到 OVERFLOW

```
MOV AX, X
CMP AX, 50
JG TOO-HIGH
SUB AX, Y
JD OVERFLOW
JNS GETRESULT
```

求补, 相当于 **NEG** AX
求负数的绝对值

```
GETRESULT:
    MOV RESULT, AX JMP JMP END-PROC
TOO-HIGH:
    ... < JMP END-PROC
OVERFLOW:
    ... < JMP END-PROC
```

END-PROC:

例. α, β 是双精度数, 分别存于 DX, AX 及 BX, CX 中, $\alpha > \beta$ 时转 L1, 否则转 L2

```
CMP DX, BX
比较2个带符号数 JG L1
JL L2
```

```
CMP AX, CX
比较2个无符号数 JA L1
```

```
L2: ... < JMP END-PROC
```

```
L1: ... < JMP END-PROC
```

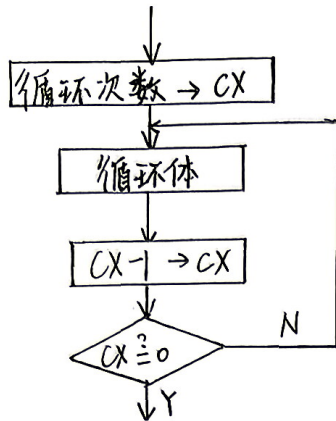
END-PROC:
...

分类	指令	转移条件	说明
(I)	JZ/JE 相等	ZF=1	为零/相等, 则转移
	JNZ/JNE	ZF=0	不为零/不相等, 则转移
	JS	SF=1	为负, 则转移
	JNS	SF=0	为正, 则转移
	JO	OF=1	溢出, 则转移
	JNO	OF=0	不溢出, 则转移
	JP	PF=1	奇偶位为 1, 则转移
	JNP	PF=0	奇偶位为 0, 则转移
	JC	CF=1	进位位为 1, 则转移
	JNC	CF=0	进位位为 0, 则转移
(II)	小于 JB/JNAE/JC	CF=1	低于/不高于等于, 则转移
	JNB/JAE/JNC	CF=0	不低于/高于等于, 则转移
	JBE/JNA	(CF V ZF)=1	低于等于/不高于, 则转移
	JNBE/JA 大于	(CF V ZF)=0	不低于等于/高于, 则转移
(III)	小于 JL/JNGE	(SF V OF)=1	小于/不大于等于, 则转移
	JNL/JGE	(SF V OF)=0	不小于/大于等于, 则转移
	JLE/JNG	((SF V OF) V ZF)=1	小于等于/不大于, 则转移
	JNLE/JG 大于	((SF V OF) V ZF)=0	不小于等于/大于, 则转移
(IV)	JCXZ CX=0	(CX)=0	CX 的内容为 0, 则转移

[注] (I) 根据条件码的值转移; (II) 比较两个无符号数, 根据比较的结果转移; (III) 比较两个带符号数, 根据比较的结果转移; (IV) 根据 CX 寄存器的值转移。

(3) 循环指令

流程:



LABEL 操作数只能使用段内直接寻址的8位位移量

LOOP 指令跳转目标 LABEL 只能在当前代码段内
且与 LOOP 指令的距离范围在 -128 ~ +127 字节

LOOP LABEL 等价于2句话:

DEC CX ; CX-1 → CX

JNZ LABEL ; CX 不为 0 就转移至 LABEL 处重新执行

为零或相等时循环指令:

LOOPZ (LOOPE) 操作数

测试条件: ZF=1 且 (CX)≠0

不为零或不相等时循环指令:

LOOPNZ (LOOPNE) 操作数

测试条件: ZF=0 且 (CX)≠0

例. 求首地址为 ARRAY 的 M 个字之和, 结果存入 TOTAL

MOV CX, M

MOV AX, 0

MOV SI, AX

AGAIN:

ADD AX, ARRAY[SI]

ADD SI, 2

DEC CX

JNZ AGAIN

LOOP AGAIN

MOV TOTAL, AX

例. 数据块传送(字节)

MOV CX, 400H

MOV SI, OFFSET SBUF; SI 指向源缓冲区 SBUF 在数据段中的起始偏移地址

MOV DI, OFFSET DBUF; DI 指向目的缓冲区 DBUF 在附加段中的起始偏移地址

AGAIN:

MOV AL, [SI] ; 从 SI 指向的地址读 1 个数据放入 AL AL 改 AX

MOV ES:[DI], AL; 从 AL 中读 1 个字节存入 DI 指向的地址 AL 改 AX

INC SI (SI)+1 → (SI) INC SI 改 ADD SI, 2

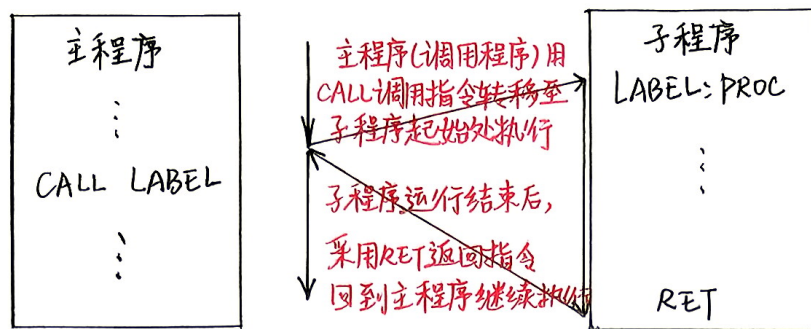
INC DI (DI)+1 → (DI) INC DI 改 ADD DI, 2

LOOP AGAIN

数据块传送(字)

400H 改 200H

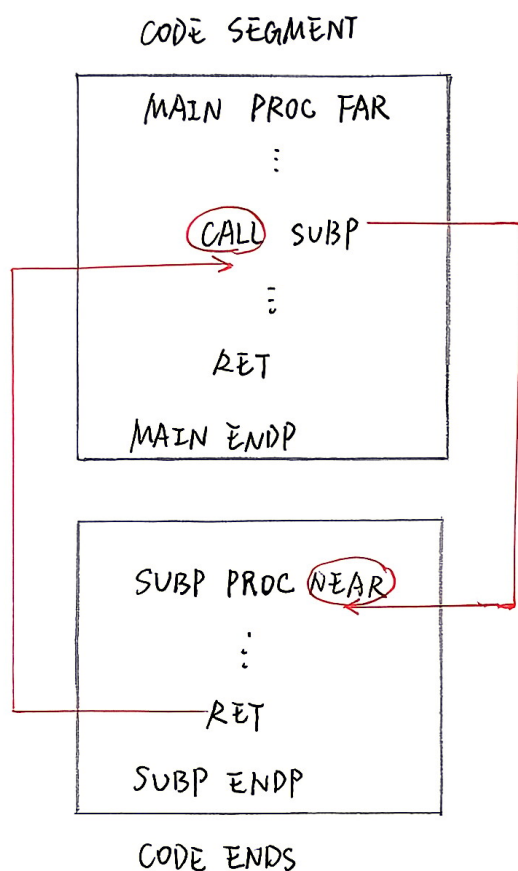
(4) 子程序调用和返回指令



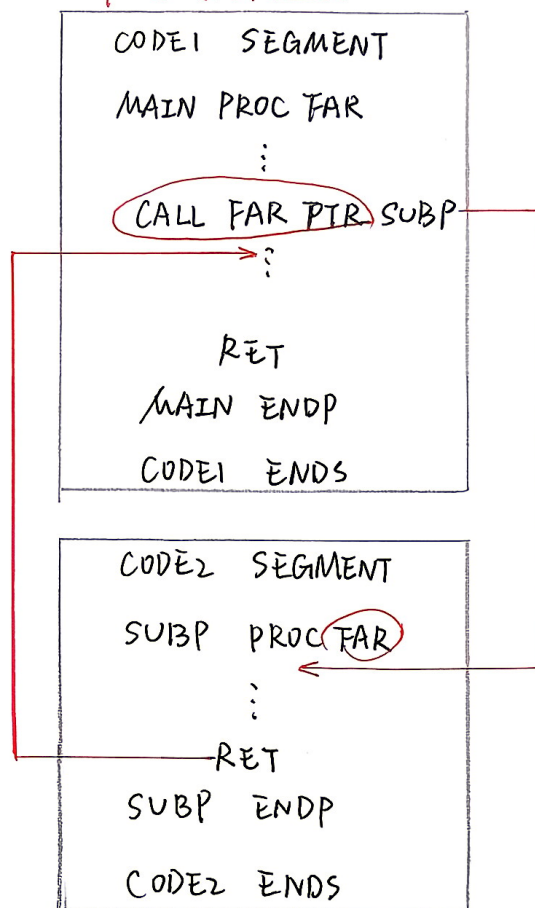
子程序: 完成特定功能的一段程序, 若某程序段在源程序内反复出现, 则可将其定义为子程序

不像转移指令有去无回, 子程序调用需要返回, 要利用堆栈保存返回地址

段内调用和返回



段间调用和返回



与 JMP 指令类似, CALL 指令分为 4 种类型:

CALL LABEL ; 段内直接调用

CALL R16/M16 ; 段内间接调用

CALL FAR PTR LABEL ; 段间直接调用

CALL DWORD PTR MEM ; 段间间接调用

段内调用: 偏移地址 IP 要入栈 (记录返回主程序后下一条执行指令的地址)

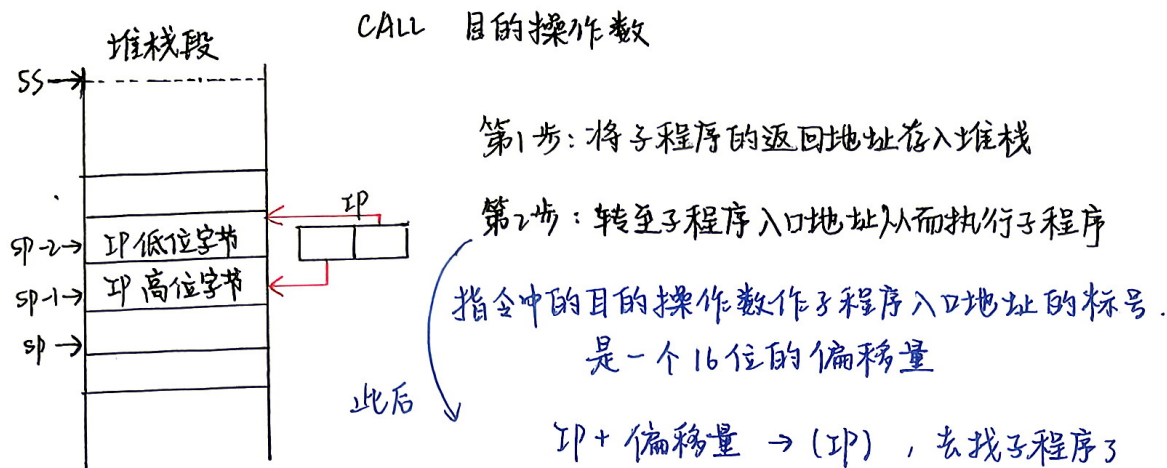
SP-2 → SP IP → SS:SP 堆栈段栈顶指针指向的地址

段间调用: 偏移地址 IP 和段地址 CS 要入栈

SP-2 → SP IP → SS:SP

SP-2 → SP CS → SS:CS

1) 段内直接调用



2) 段内间接调用

与段内直接调用的区别是:

子程序的入口地址的寻址方式不同,

可以是寄存器操作数和各种寻址方式的存储器操作数,

但不可以是立即数、段寄存器操作数

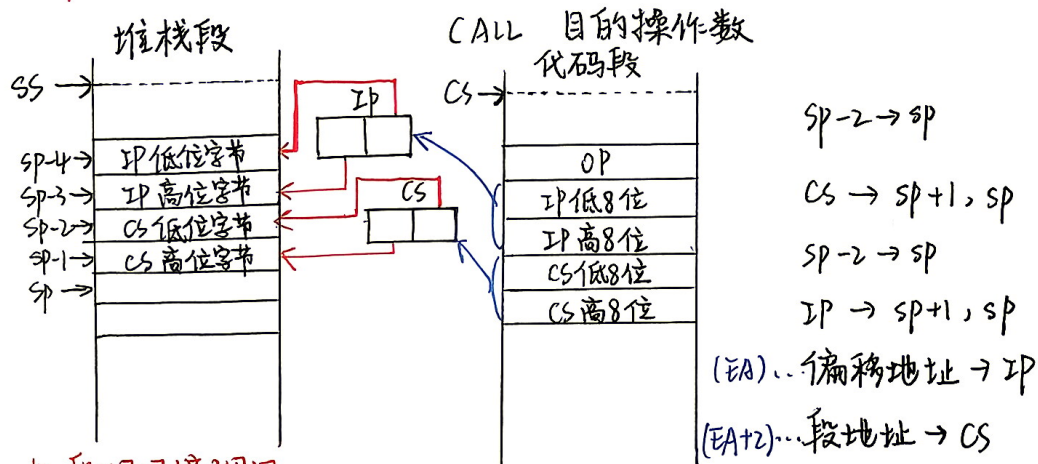
CALL BX

BX 内容是子程序偏移量

CALL WORD PTR [BX]

BX 所指内存单元值是子程序偏移量

3) 段间直接调用



4) 段间间接调用

与段间直接调用的区别是:

子程序的入口地址的寻址方式不同

可以是寄存器操作数和各种寻址方式的存储器操作数

CALL DWORD PTR [BX]

BX 所指内存双字单元的值是子程序的偏移量和段值

返回指令

1) 段内近返回 RET

$(sp)+1, (sp) \rightarrow (ip)$

$(sp)+2 \rightarrow (sp)$

2) 段内带立即数返回 RET EXP

$sp + EXP \rightarrow sp$

使程序可方便废除若干执行CALL指令前的入栈参数

3) 段间远返回 RET

$(sp)+1, (sp) \rightarrow (ip)$

$(sp)+2 \rightarrow (sp)$

$(sp)+1, (sp) \rightarrow (cs)$

$(sp)+2 \rightarrow (sp)$

4) 段间带立即数远返回 RET EXP



CODE SEGMENT

MAIN PROC FAR

:

PUSH AX

PUSH BX

PUSH CX

CALL SUB

:

RET

MAIN ENDP

SUB PROC NEAR

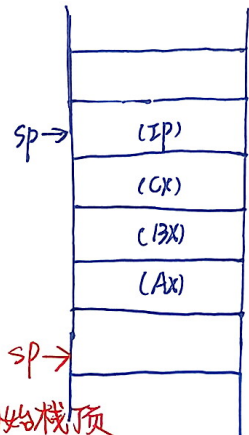
:

RET 6

SUB ENDP

CODE ENDS

堆栈段



恢复到初始栈顶

例. 十进制转换为ASCII码的子程序

∵ 数字 0~9 的 ASCII 码是 30H~39H ∴ +30H

∵ 字母 A~F 的 ASCII 码是 41H~46H ∴ +37H

主程序:

MOV DL, 28H; (DL) = 00101000H

CALL HTOASC

子程序:

HTOASC PROC

AND DL, 0FH ; 保留低4位

OR DL, 30H ; 置高4位为3

CMP DL, 39H ; 若 ≤ 39. 即 30~39

JBE HTOEND

ADD DL, 7H

HTOEND:

RET

HTOASC ENDP

1.5 中断及中断返回指令

中断源: 引起紧急事务的事件

中断服务/处理程序: 处理紧急事务的程序

中断是一种改变程序执行顺序的方法

8088 CPU支持256个中断, 每个中断用一个编号(中断向量号)区别

中断指令有3条: INT TYPE IRET INTO

掌握了中断调用, 进而能掌握系统功能调用

中断向量表是一个特殊的线性表, 保存系统所有中断服务程序的入口地址

(偏移量、段地址), 有256个元素(0~0FFH),

每个元素占4字节, 共1K字节

若中断号为N, 则中断向量表中存储该中断处理程序的入口地址的单元地址是4N

1) 中断指令INT

INT TYPE

立即数TYPE是一个0~0FFH范围内的整数

第1步: 标志寄存器压栈, 清除标志位IF和TF

第2步: 把CS和IP内容压栈

第3步: $(TYPE \times 4) \rightarrow (IP)$ $(TYPE \times 4 + 2) \rightarrow (CS)$

3) 中断返回指令IRET

IRET

中断服务程序执行完毕时, CPU恢复被中断的现场, 返回引起中断的程序

第1步: 从栈顶弹出内容送入IP

第2步: 从新栈顶弹出内容送入CS

第3步: 从新栈顶弹出内容送入标志寄存器

2) 溢出指令INTO

INTO

标志位OF为1时引起中断, 影响标志位IF、TF

中断与子程序的比较?

相同: 暂停当前程序的执行转而执行另一程序段, 当程序段执行完时, CPU自动恢复原程序执行

不同: 子程序调用可知, 中断源依自身需要产生的中断不可预见(用INT指令引起的除外)

子程序调用由CALL指令实现, 但没有调用中断的指令, 只有发出中断请求的事件

子程序的返回用RET, 中断服务程序的返回用IRET

子程序由应用系统开发者编写而中断服务程序由系统软件设计者编写

⑥ 处理机控制类指令

(1) 标志处理指令 (只影响本指令指定的标志)

CLE 进位位CF置0

CLD 方向标志位DF置0

CLI 中断标志位IF置0

STC 进位位CF置1

STD 方向标志位DF置1

STI 中断标志位IF置1

CMC 进位位CF求反

(2) CPU控制类指令

1) 空操作指令 NOP

占一个字节存储单元, 空耗一个指令执行周期

2) 暂停指令 HLT

等待中断信号时, 该指令使CPU处于暂停工作状态

3) 换码指令 ESC

把浮点指令交给浮点(协)处理器执行

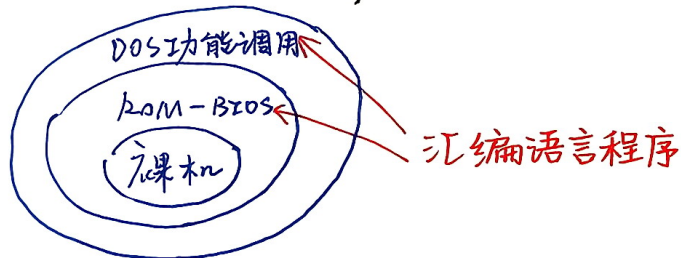
4) 等待指令 WAIT (不影响任何标志位)

使CPU处于等待状态直至协处理器完成运算, 并用重启信号唤醒CPU

5) 封锁数据指令 LOCK

作指令前缀, 可置于任何指令前

该前缀使该指令执行时间内, 实现总线封锁, 不让其它控制器控制总线



80x86 提供了 2 种类型的例行程序供用户调用:

基本输入输出系统 BIOS

磁盘操作系统 DOS

DOS 中断 21H 的功能 5 与 BIOS 中断 17H 的功能 0 等效

但 BIOS 中断 17H 的功能 2 为读打印机状态, 它无等效 DOS 功能

系统功能调用步骤:

在 AH 中设置系统功能调用号

在指定寄存器中设置入口参数

用中断调用指令 (INT 21H) 执行功能调用

依出口参数分析功能调用的执行情况

DOS 利用 21H 号中断提供近 100 个系统功能, 主要包括设备管理、目录管理、文件管理

DOS 常用输入输出功能调用 (INT 21H)

功能号	功能	调用参数	返回参数
AH=01H	键盘输入		AL=输入字符
AH=02H	显示器输出	DL=输出显示的字符	
AH=05H	打印机输出	DL=打印字符	
AH=09H	输出字符串	DS:DX=字符串地址 ↑ 存字符串在数据段中的偏移地址	显示字符串在内存中的首地址
AH=0AH	输入字符串	存数据段的段地址 DS:DX=缓冲区地址	
AH=0BH	检验键盘状态		AL=FFH, 有输入; AL=00H, 无输入

DS:DX 是逻辑地址的形式

ASCII 码中 0~31 是控制字符, 不会在屏幕显示但会触发蜂鸣器

响铃:

MOV DL, 07H

MOV AH, 02H

INT 21H

返回操作系统:

MOV AH, 4CH

INT 21H

结束当前正在执行的程序并返回 OS,

屏幕显示 OS 提示符 C>

例. 当原始数据为任意数据时, 一般用键盘输入方法, 调用DOS 21H中断:

01H号DOS功能调用

MOV AH, 01H
INT 21H

只管读取就好, 无需赋值

当CPU执行到这2条语句后就等待键盘输入字符, 键入的字符以ASCII码形式送入AL寄存器, 如键入'3', 则送入AL=33H

例. 显示一个"?"

02H号DOS功能调用

MOV AH, 02H ; 功能调用号
MOV DL, '?' ; 入口参数 (ASCII码)
INT 21H ; 功能调用

ASCII码 0DH 0AH

例. 回车换行的子程序

CRLF PROC

PUSH AX

PUSH DX

} 保护寄存器

MOV AH, 02H

MOV DL, 0DH

INT 21H

MOV AH, 02H

MOV DL, 0AH

INT 21H

POP DX

POP AX

} 恢复寄存器

RET

CRLF ENDP

例. "按任意键继续"提示

09H号DOS功能调用

MSGKEY DB 'Press any key to continue...', '\$' (不显示)

MOV AH, 09H

MOV DX, OFFSET MSGKEY ; 将字符串MSGKEY在DS段中的偏移地址存入DX

INT 21H

结合DS设置:

使DS指向数据段
(不能直传立即数, 要用通用寄存器中转)

MOV AX, DATA
MOV DS, AX

切不可"MOV DS, DATA", 不支持立即数寻址

例. DOS 21H 功能的 0BH 可检验一个键是否被按动:

是, 则 AL 被放入 0FFH

否, 则 AL = 00H

Sounder:

MOV DL, 07H

MOV AH, 02H

INT 21H

MOV AH, 0BH

INT 21H

loop?

INC AL

AL 被置 0FFH (按下) 或 00H (没按下)

JNZ Sounder

RET

不断循环执行直至按下键, 才退出循环并返回

(屏保)